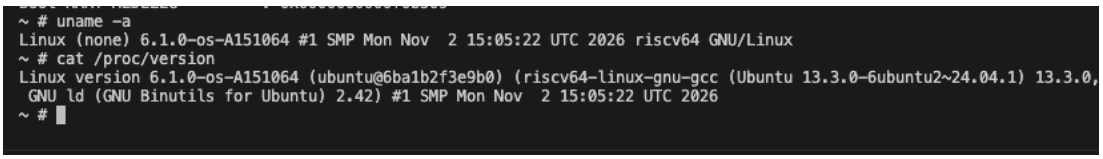
Assignment I - Compiling a Custom Linux Kernel & Implementing New System Calls

謝文珈 A151064

I. Compiling a custom Linux kernel

A. screenshot of the outputs of uname -a and cat /proc/version



B. steps I took to compile the kernel

Step 1: Environment Setup & Toolchain Verification

To ensure a reproducible build environment isolated from host contamination, compilation was performed inside a Docker container (Ubuntu 24.04):

```
# 在主機 HW1-student 目錄啟動容器
docker compose up -d --build
# 透過生成的 RSA 私鑰與指定連接埠 22025 連線登入：
ssh -i ./docker/home/ubuntu/.ssh/id_rsa -p 22025 ubuntu@localhost
```

Verified that the 64-bit RISC-V GNU cross-toolchain and QEMU emulator were properly installed:

```
riscv64-linux-gnu-gcc -v
qemu-system-riscv64 --version
```

Step 2: Kernel Source Acquisition

Acquired the official Linux kernel tree tagged at release v6.1 (CIP Super-LTS release). Used --depth 1 to skip unnecessary historical commit data, reducing download size from ~5 GB to ~300 MB:

```
cd /home/ubuntu
git clone https://github.com/torvalds/linux --branch v6.1 --depth 1 linux
cd /home/ubuntu/linux
```

Step 3: Kernel Configuration (.config) & Local Version Setup

Target Architecture & Base Configuration: Generated the baseline .config file targeting the RISC-V 64-bit architecture (ARCH=riscv) using the GNU cross-

compiler prefix (CROSS_COMPILE=riscv64-linux-gnu-):

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig
```

Custom Local Version Configuration (CONFIG_LOCALVERSION): Injected the mandatory student ID suffix and disabled Git commit hash appending:

```
# Set custom student suffix: -os-A151064
scripts/config --file .config --set-str CONFIG_LOCALVERSION "-os-A151064"
# Disable automatic Git SHA appending to prevent version mismatch
scripts/config --file .config --disable CONFIG_LOCALVERSION_AUTO
# Re-evaluate dependencies and generate headers
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- olddefconfig
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- prepare
```

Release String Verification: Verified that the kernel release string exactly matched the grading specification:

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- kernelrelease
```

Output:

```
6.1.0-os-A151064
```

Step 4: Compiling the RISC-V Kernel Image

Compiled the uncompressed binary kernel image (Image) using all available CPU threads (-j$(nproc)):

```
make -j$(nproc) ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- Image
```

Compilation Duration: ~4 minutes on an 8-core CPU.

Output Artifact: Generated at arch/riscv/boot/Image (~21 MB).

Successful Output:

```
  LD          vmlinux
  NM          System.map
  SORTTAB vmlinux
  OBJCOPY arch/riscv/boot/Image
  Kernel: arch/riscv/boot/Image is ready
```

Step 5: Incorporating Custom System Calls (Incremental Compilation)

After implementing sys_revstr and sys_tempbuf:

1. Updated arch/riscv/include/uapi/asm/unistd.h (added syscall 451 & 452).
2. Declared prototypes in include/linux/syscalls.h.
3. Added compilation targets obj-y += sys_revstr.o sys_tempbuf.o in kernel/Makefile.
4. Created kernel/sys_revstr.c and kernel/sys_tempbuf.c.
5. Re-executed incremental compilation:

```
cd /home/ubuntu/linux
make -j$(nproc) ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- Image
```

Step 6: Testing & Booting with QEMU

1. Statically cross-compiled user-space test binaries (test_revstr, test_tempbuf, verify_syscalls) using riscv64-linux-gnu-gcc -static.
2. Repacked the test binaries into initramfs.cpio.gz.
3. Booted the custom kernel inside QEMU:

```
cd /home/ubuntu
riscv64-linux-gnu-gcc -static -o test_revstr test_revstr.c
riscv64-linux-gnu-gcc -static -o test_tempbuf test_tempbuf.c
riscv64-linux-gnu-gcc -static -o verify_syscalls verify_syscalls.c

# Inject test binaries into initramfs
mkdir -p /home/ubuntu/initramfs && cd /home/ubuntu/initramfs
gzip -dc ../initramfs.cpio.gz | cpio -idmu
cp ../test_revstr .
cp ../test_tempbuf .
cp ../verify_syscalls .
find . -print0 | cpio --null -o -H newc | gzip -c > ../initramfs.cpio.gz

# Boot RISC-V virtual machine in QEMU
cd /home/ubuntu
qemu-system-riscv64 -nographic -machine virt \
    -kernel linux/arch/riscv/boot/Image \
    -initrd initramfs.cpio.gz \
```

```
-append "console=ttyS0 loglevel=3"
```

Verified successful kernel boot and custom version inside QEMU BusyBox shell

via uname -a and cat /proc/version.

II. Implementing new system calls

A. The execution result of provided test_revstr user program for sys_revstr



B. The dmesg output of sys_revstr



C. The execution result of provided test_tempbuf user program for sys_tempbuf



D. The dmesg output of sys_tempbuf



E. how I added system calls to my custom Linux kernel and the implementation details of sys_revstr and sys_tempbuf

RISC-V 在 Linux v6.1 中採用通用 UAPI 定義。我們共修改了三個檔案

1. **arch/riscv/include/uapi/asm/unistd.h**：
   a. 定義系統呼叫編號 #define __NR_revstr 451 與 #define __NR_tempbuf 452。
   b. 透過架構巨集 __SYSCALL(__NR_revstr, sys_revstr) 註冊分派常式。
   c. 將系統呼叫上限 #define __NR_syscalls 453 更新，防止呼叫時發生邊界越界。
2. **include/linux/syscalls.h**：
   在標頭檔中宣告 asmlinkage long sys_revstr(...) 與 sys_tempbuf(...) 函數

原型，使符號在整個核心編譯單元可見。

3. **kernel/Makefile**：

   加入 obj-y += sys_revstr.o sys_tempbuf.o，指示 Kbuild 編譯器將 kernel/sys_revstr.c 與 kernel/sys_tempbuf.c 編入核心二進位映像 (vmlinux)。

Implementation details of sys_revstr

1. **使用者與核心記憶體隔離**：

   使用者指標 char __user *str 位於未授權的分頁表，直接 dereference 會觸發 SMAP/SMEP 硬體保護或造成 Kernel Panic。

   使用 copy_from_user(kstr, str, n) 安全拷貝至核心 heap，反轉完成後使用 copy_to_user(str, kstr, n) 寫回。

2. **原地反轉演算法 (In-place Reversal)**：

   採用對稱雙指針演算法，迴圈執行 $n/2$次對調 kstr[i] 與 kstr[n - i - 1]，時間複雜度為 $O(n)$，且不需額外配置第二份字串緩衝區。

3. **記憶體管理與安全防護**：

   分配 n + 1 位元組並手動補上 \0 null-terminator，避免 pr_info 發生 Buffer Over-read。

   採用 goto out; 結構化清理路徑，保證無論在成功或複製失敗時都會呼叫 kfree(kstr)，絕無記憶體洩漏。

Implementation details of sys_tempbuf

1. **核心雙向鏈結串列 (struct list_head)**：

   a. 節點定義採用 C99 Flexible Array Member：

```
struct hw1_tempbuf_record {
        struct list_head list;
        size_t len;
        char text[];
};
```

   b. 使用 struct_size(record, text, size + 1) 一次性配置結構體與字串本體。

2. **併行保護 (DEFINE_MUTEX)**：

   使用 mutex_lock(&hw1_tempbuf_lock) 保護所有串列變更與走訪，避免

多行程或核心搶占 (preemption) 導致鏈結串列指針損壞。

3. **三種模式之實作機制：**
   a. **ADD (1)**：使用 list_add_tail() 將節點插入至全域串列尾端，並以 pr_info 記錄。
   b. **REMOVE (2)**：使用 list_for_each_entry_safe() 安全走訪，精確比對長度與內容；**僅刪除第一筆匹配項**（以 list_del 與 kfree 釋放）；若全部走訪完皆無匹配，回傳 -ENOENT。
   c. PRINT (0)：
      1. Pass 1：走訪計算總長度，使用 check_add_overflow() 防止整數溢位。
      2. Pass 2：動態分配 kmalloc，以單一空格相連格式化字串，並呼叫 pr_info("[tempbuf] %s\n", joined)。
      3. 回傳值處理：複製最多 size - 1 字元至使用者緩衝區並在末端寫入 \0，回傳實際複製字元數（嚴格排除結尾的 \0）。

4. **錯誤代碼規範**：
   a. data == NULL 或 size == 0：回傳 -EFAULT。
   b. 不合法模式（非 0, 1, 2）：回傳 -EINVAL。
   c. 核心記憶體配置失敗：回傳 -ENOMEM。